

웹툰 인사이트

Text analysis to confirm trends of webtoon

TEAM 1조 | ALPHA911



김예선 문영식 조인준 최희범

01

INTRO

개요

02

PROCESS

수행 절차

03

PART

역할 분담

04

RESULT

수행 결과

05

ASSESSMENT

자체 평가 및 의견

프로젝트 개요

TEAM |
ALPHA911

분석할 데이터가 충분한지

분석 실현이 가능한지

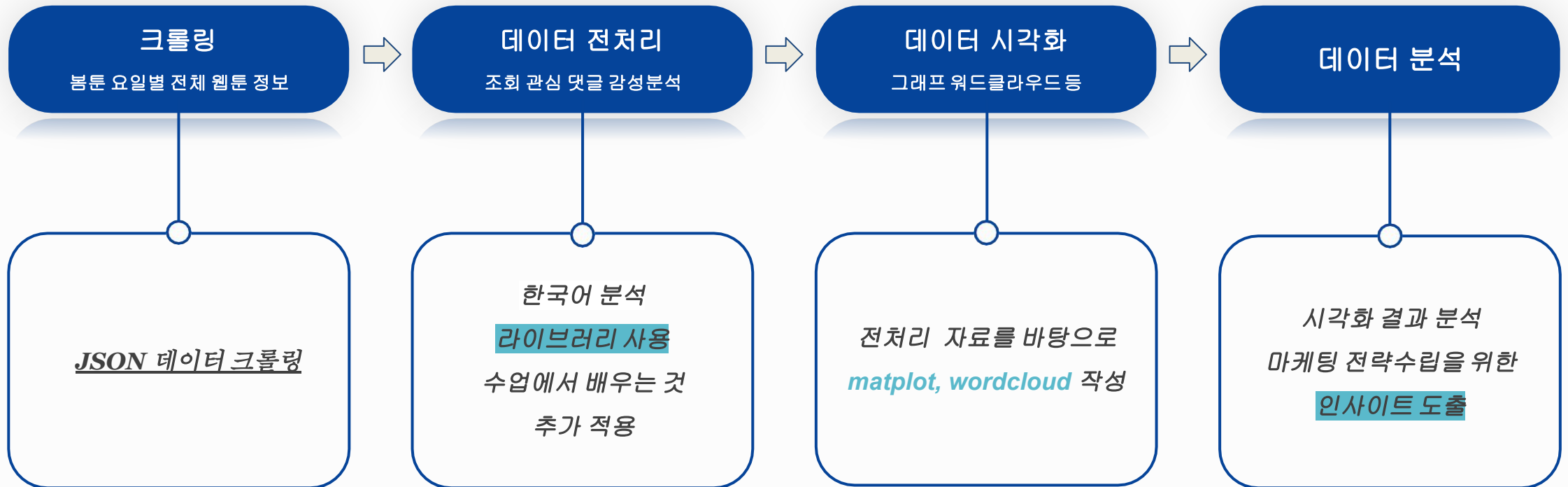
분석 후 인사이트가 있을지

심화 학습이 가능한지



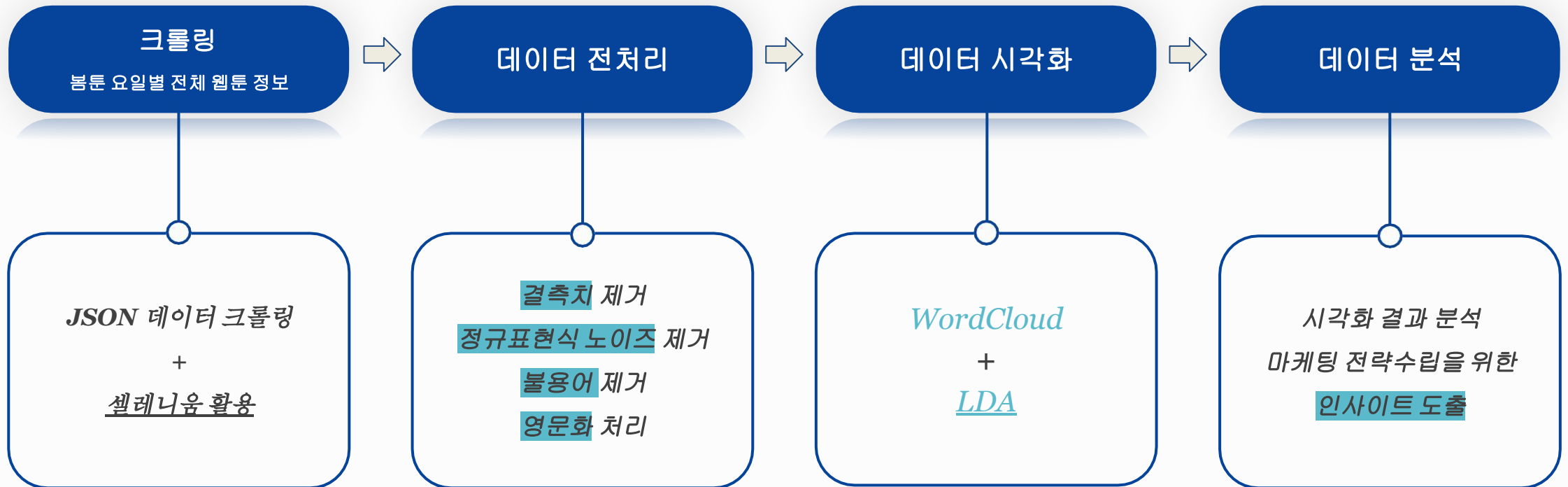
프로젝트 수행절차

프로젝트 예상 시나리오



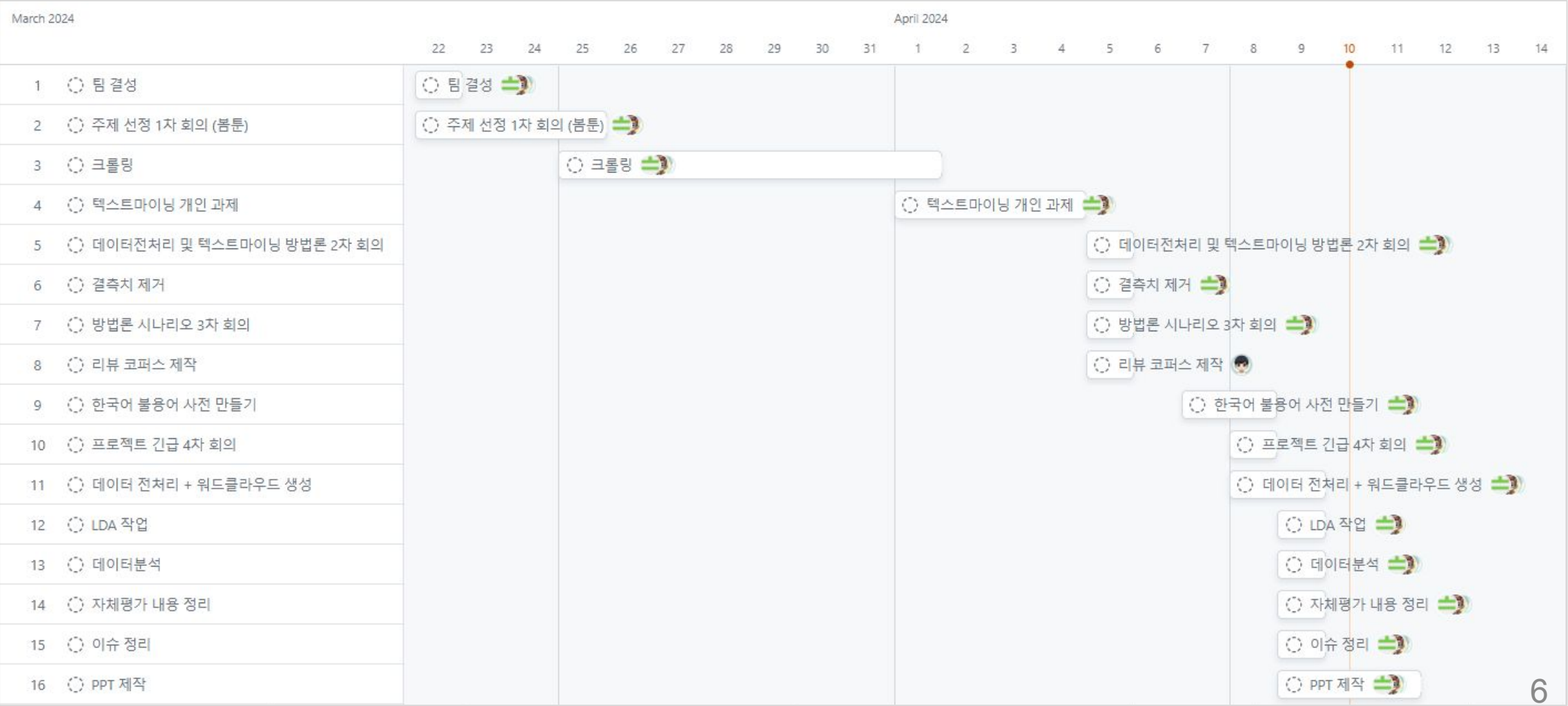
프로젝트 수행절차

프로젝트 실제 시나리오



프로젝트 수행절차

프로젝트 수행 WPT(Work Plan Timeline)



프로젝트 수행절차

프로젝트 수행 WPT(Work Plan Timeline)

구분	기간	활동	비고
팀 결성	3/22(금)		
주제선정 1차 회의 (봄툰)	3/22(금) ~ 3/25(월)	프로젝트 주제선정, 기획안 정리	다수결 주제선정
크롤링	3/25(월) ~ 4/1(월)	‘봄툰’ 댓글 크롤링 및 코드 작성	Json Request, Selenium
텍스트마이닝 개인과제	4/1(월) ~ 4/4(목)	개인과제 수행	프로젝트 적용 방안 고찰
데이터전처리 및 텍스트마이닝 방법론 2차회의	4/5(금)	댓글 결측치 제거 후 데이터전처리 방안 협의	
결측치 제거	4/5(금)	댓글 결측치 제거	
방법론 시나리오 3차회의	4/5(금)	워드클라우드 작성방안 논의	
리뷰 코퍼스 제작	4/5(금)	댓글 코퍼스 생성	
한국어 불용어 사전 만들기	4/7(일) ~ 4/8(월)	불용어 데이터 개별수집 및 코퍼스 제작	
프로젝트 긴급 4차 회의	4/8(월)	프로젝트 결과물 가공 및 역할분담	(한/영)워드클라우드, PPT초안 작성
데이터 전처리 + 워드클라우드 생성	4/8(월) ~ 4/9(화)	데이터 정규화, 형태소 분석기별 비교	OKt, Hannanum, Mecab, Pororo, Khaii
LDA 작업	4/9(화)	LDA 작성 및 코드 개선	
데이터분석	4/9(화)	워드클라우드, LDA 인사이트 분석	
자체평가 내용 정리	4/9(화)	프로젝트 자체평가 내용 정리	
이슈정리	4/9(화)	개발과정 발생 이슈 문서화	주제선정, 크롤링, 전처리 등
PPT 제작	4/9(화) ~ 4/11(목)	PPT 템플릿 작성, 자료취합, 레이아웃 조정	
총 개발기간 (총 3주)	3/22(금) ~ 4/11(목)		

프로젝트 팀 구성 및 역할

TEAM | ALPHA911



👑 김예선

웹크롤링

데이터전처리

워드 클라우드 작성

PPT작성

대본 작성



👑 문영식

Git repo/project 관리

웹크롤링

데이터 전처리

워드 클라우드 작성

데이터 LDA 처리

산출물 관리

PPT 작성

대본 작성



👑 조인준

웹크롤링

데이터 전처리

데이터 영문 번역처리

영문 전처리

영문 워드클라우드 작성

영문 데이터 LDA 처리

PPT 작성



👑 최희범

웹크롤링

데이터전처리

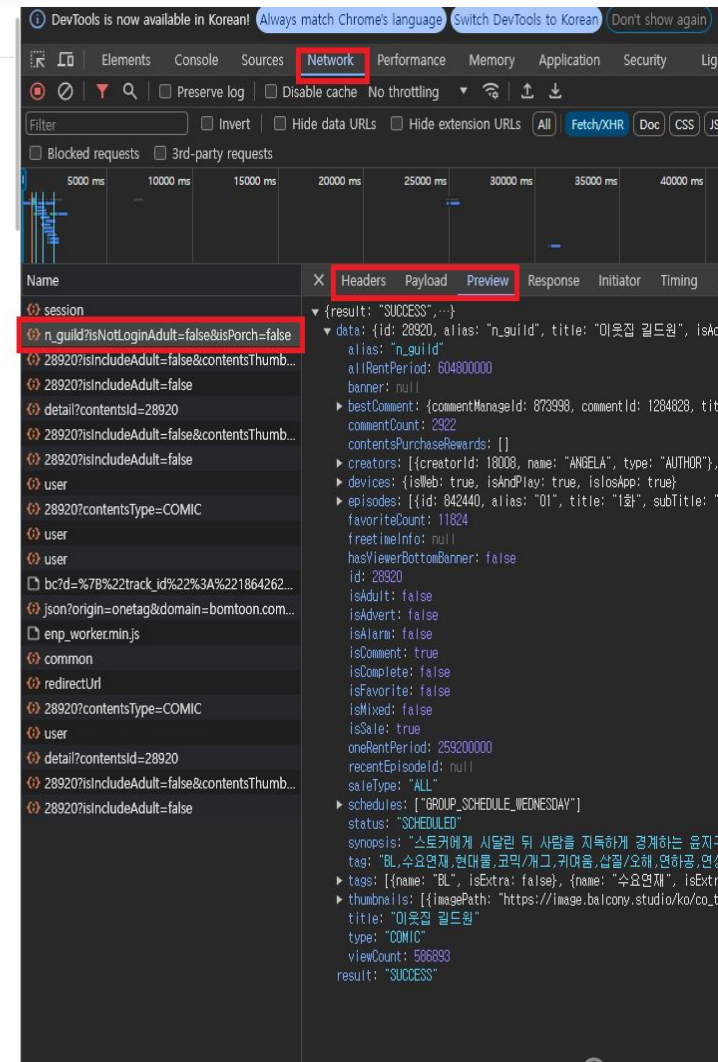
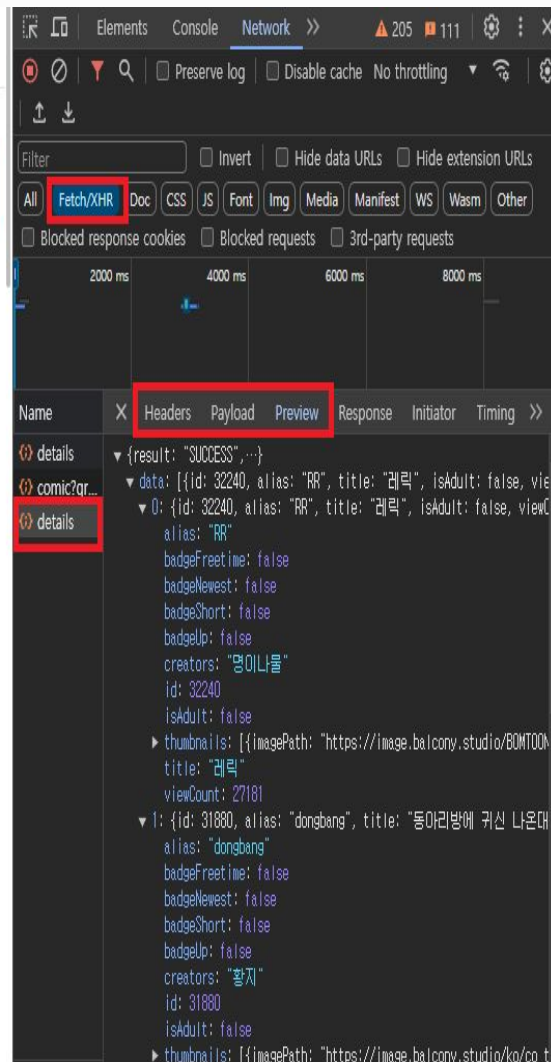
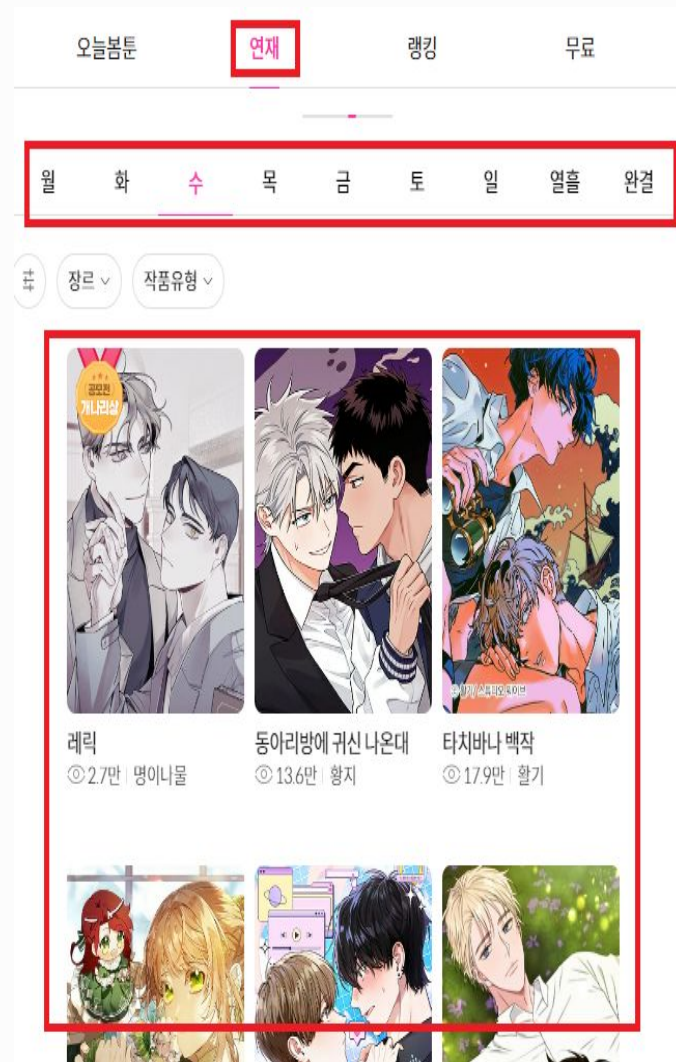
워드 클라우드 작성

데이터 LDA 처리

PPT작성

프로젝트 수행 결과

봄툰 웹 분석



CommentCrawling BUM

CommentCrawling Jo.ipynb

CommentCrawling Kim

CommentCrawling Moon

00_CommentCrawling_Main.ipynb

AllReview Jo.csv

FridayReview.csv

MondayReview Jo.csv

SaturdayReview Jo.csv

SundayReview Jo.csv

TenDaysReview Jo.csv

ThirsdaysReview Jo.csv

TuesdayReview Jo.csv

WednesdayReview Jo.csv

```
# 1단계 : 해당 도메인의 툰 정보를 얻기위해선 제일 먼저 요일별 제공하는 툰 고유 id를 알아야한다는 특성이 있어서 id부터 스크랩한다.
weeks = ('MONDAY', 'TUESDAY', 'WEDNESDAY', 'THURSDAY', 'FRIDAY', 'SATURDAY', 'SUNDAY')

db = {'toons_ids': [],
      'reviews_list': []}

for week in tqdm(weeks, desc='각 요일 돌면서 전체 만화 id 스크랩'):
    url = f'https://www.bomtoon.com/api/balcony-api-v2/contents/tab/schedule/comic?groupMenu={week}&isIncludeTen=false&sort=POPULAR&adultToggle=false'
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36',
        'X-Balcony-Id': 'BOMTOON_COM',
        'Content-Type': 'application/json'
    }
    resp = requests.request("GET", url, headers=headers)
    jsn = json.loads(resp.text)
    lens = len(jsn['data'])
    for l in range(lens):
        db['toons_ids'].append(jsn['data'][l][1])

# 2 단계 : 툰ids를 기반으로 툰 상세정보를 스크랩한다.
url = f'https://www.bomtoon.com/api/balcony-api-v2/contents/tab/details'
contentsIdsid = db['toons_ids']
contentsIdsid = ','.join(map(str, db['toons_ids']))
# print(contentsIdsid)
payloads = json.dumps({
    "contentsIds": contentsIdsid, #알고싶은 toon id들이 이곳에 들어오면 됨.
    "contentsThumbnailType": "VERTICAL,MAIN,SQUARE"
})

resp = requests.request("POST", url, headers=headers, data=payloads)
jsn = json.loads(resp.text)
lens = len(jsn['data'])
for l in tqdm(range(lens), desc="리뷰 수집 중"): # 실제로 불러와진 툰 정보 갯수만큼 반복 수행.
    # for l in tqdm(range(10), desc="리뷰 수집 중"): # 테스트
    title = jsn['data'][l][1]['title'] # 툰제목 : 댓글 할 예정
    alias = jsn['data'][l][1]['alias'] # 각 툰 댓글 페이지 들어가려면 툰 alias가 필요
    toon_url = f'https://www.bomtoon.com/api/balcony-api-v2/contents/{alias}?isNotLoginAdult=false&isPorch=false'
    toon_resp = requests.request("GET", toon_url, headers=headers)
    toon_jsn = json.loads(toon_resp.text)
    commentCount = toon_jsn['data']['commentCount'] # 댓글 수집을 하려면 갯수 정보가 있어야 함.
    if commentCount != 0: #댓글이 있는지 확인하고 있으면 수집 없으면 "noreviewtoon."
```

프로젝트 수행 결과

크롤링 결과 데이터

작가명	작가ID	댓글총갯수	댓글총페이지	게시물인덱스	댓글쓴날짜	댓글의id	작품명	작품회차	댓글자명	댓글	좋아요횟수
반들	321	20	11	12.0	2023-11-23 09:54	1508184.0	3과 2분의	1화	봄밤달빛	다들 비주얼 합격!	2.0
반들	321	20	11	18.0	2024-03-18 07:48	1679736.0	3과 2분의	1화	윤희겨미	첫화부터 흥미진진.기대된다.	1.0
반들	321	20	11	19.0	2023-11-24 09:40	1509237.0	3과 2분의	1화	밤에 몰래	아따마. 흥미진진하다	1.0
반들	321	20	11	3.0	2023-11-23 14:14	1508294.0	3과 2분의	1화	그럴수도	삼각구도 좋구나	1.0
반들	321	20	11	6.0	2023-11-22 21:27	1507632.0	3과 2분의	1화	봄툰러3cf	엄청난 삼각관계	1.0
반들	321	20	11	3.0	2023-11-22 21:31	1507637.0	3과 2분의	2화	봄툰러3cf	ㅋㅋㅋㅋㅋㅋ아미친... 덕질메이트 찾았네 너무 귀엽다	13.0
반들	321	20	11	14.0	2023-11-24 23:17	1509743.0	3과 2분의	2화	메인공은	야 이런 삼각구도 참봐 ㅋㅋㅋㅋ 애들 넘귀엽다	6.0
반들	321	20	11	7.0	2023-11-25 10:47	1510077.0	3과 2분의	2화	ㅎㅎㅎ	덕질메이트 ㅎㅎㅎ	4.0
반들	321	20	11	2.0	2023-11-28 22:43	1512493.0	3과 2분의	2화	가즈아	미친 ㅋㅋㅋㅋㅋㅋㅋㅋㅋ 덕메찾았네 ㅋㅋㅋㅋㅋㅋ 나도 초대해줘 같이 빨게 ㅋㅋㅋ	3.0
반들	321	20	11	11.0	2023-11-23 22:53	1508789.0	3과 2분의	2화	점	나두 끼워 애두라 나도 태경이 혼자 좋아하고 있는 거 같아	2.0
반들	321	20	11	3.0	2024-01-16 14:30	1618674.0	3과 2분의	2화	봄툰러024	나도 하태경 좋아해 부려고 계속 돌아와 계속	1.0

	댓글
0	아니.... 상상도 못한 방법으로 회피하셔가지고 저도 놀랐어요...
1	아무리 생각해도 어릴적에 너무나도 매력적인 남성과의 접촉이,,,, 그의 성적 취향을...
...	...
87780	봄툰은 연재빠르는데 카카오페이지는 진짜 안올라오네ㅠㅠㅠ... 이용권 다 바꿔놨는데
87781	오~~~♡♡근데 무료가 더있었으면....

73335 rows x 1 columns

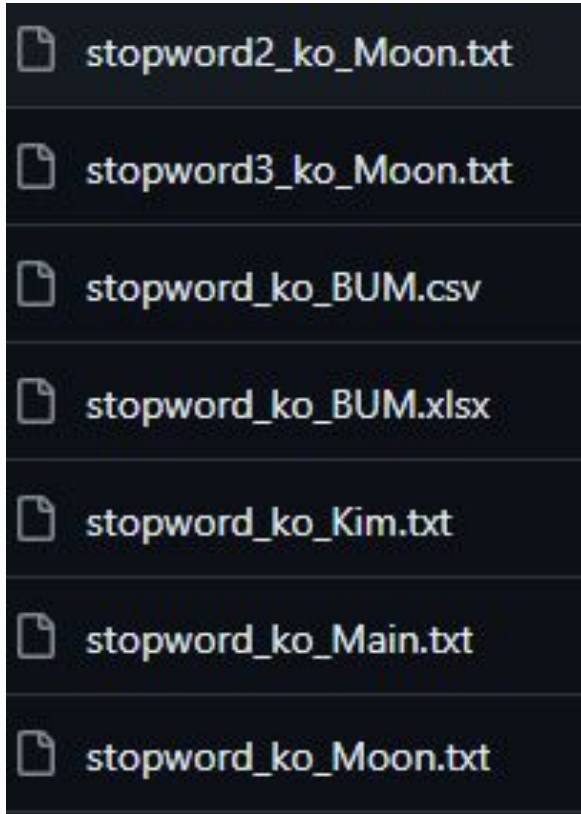
#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Unnamed: 0	89489 non-null	float64
1	작가명	89496 non-null	object
2	작가ID	89496 non-null	object
3	댓글총갯수	89496 non-null	object
4	댓글총페이지	89496 non-null	object
5	게시물인덱스	85146 non-null	object
6	댓글쓴날짜	85146 non-null	object
7	댓글의idx	85146 non-null	object
8	작품명	85146 non-null	object
9	작품회차	82156 non-null	object
10	댓글자명	85138 non-null	object
11	댓글	73335 non-null	object
12	좋아요횟수	85146 non-null	object

```
import pandas as pd

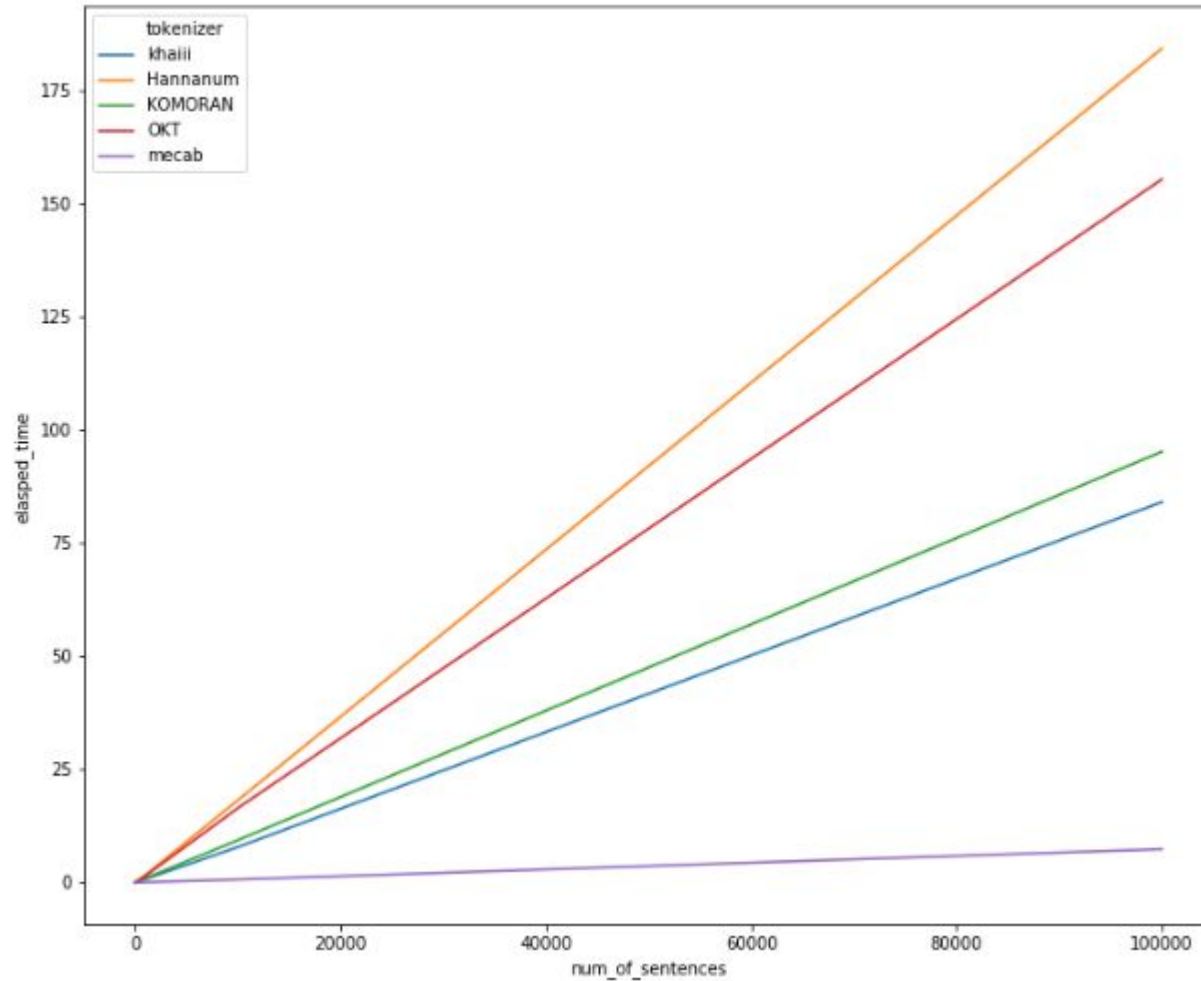
df1 = pd.read_csv('/content/리뷰데이터네임.csv')
df1.info()
df1.isna().sum()
df1.isnull().sum() # 결측치
df1 = df1.dropna(subset = ['댓글']) # 결측치 있는 댓글 행 제거
df1['댓글']
df1.isnull().sum() # 결측치
df1.info()
df1
df1.to_csv("AllReviewMVDrop BUM.csv", index = False)
df1.to_excel("AllReviewMVDrop BUM.xlsx", index = False)

review_data = pd.read_csv('/content/boom_final.csv') #결측치 제거된 데이터 업로드 후 읽음 "boom"
review_data.head() #불러온 데이터 빠르게 확인
print('데이터 개수: {}'.format(len(review_data)))
# 문자열 아닌 데이터 모두 제거
review_data = [review for review in review_data['댓글'] if type(review) is str]
# 리스트를 DataFrame으로 변환
review_df = pd.DataFrame({'댓글': review_data})
# csv 파일로 내보내기
review_df.to_csv('AllReviewCorpus.csv', index=False)
# XLSX 파일로 내보내기 (openpyxl 라이브러리가 필요할 수 있습니다)
review_df.to_excel('AllReviewCorpus.xlsx', index=False)
```

```
no_review_data.csv
파일 편집 보기
cleaned_review
상상도 못한 방법으로 회피하셔가지고 저도 놀랐어요
아무리 생각해도 어릴적에 너무나도 매력적인 남성과의 접촉이 그의 성적 취향을 결정짓게 만든것이 아닐까 합니다
하계란아무지개였은미니토스트아이팟케이스쓰는공어때
아저씨가 평생 책임져요
죄송한데 한번만 실수로 벽에 끼이셨으면
뽀갈하고 이마깨지기큐
문성이 가슴에 있는 햄버거 먹으면 되겠네
좋은 독자가 될게요
과거의 옆집 아저씨가 지금은 자신을 선생님이라고 부르고 있다니
아기공 동정따먹고 감옥가도 할말없다는 연상수
가슴이 시끄럽네요
너는 싸우겠다는 말을 뽀스만 입고 하니
진심 미칠거같아 총 안맞을 수 있는데 공 차에 안타서 맞은거임퓨
수 허벅지랑 공 허리가 눈물이 입가에서 흐른다
거대 날다람쥐야
저런 엄한 몸매로 밥을 해주다니 결혼이쥬
술 약한 아기공 집에 데려다 줘 떨어지는 라면 봉지 잡아 줘 연하의 키스는 피하는문성씨 죄가 많네
멀끔하고 가슴을 내미는데 고백을어떻게거 절합니까 여우수가 분명합니다
와시 허리들어간거봐 옷 바람직하다
문성씨 시계 찬거 풀림 싹 댄디 싹떡대 싹 간지남 칼리는 같은 느낌을 배로 주는 같아서 미치겠음
먼곳에 가있었다는 과거였던걸까요 미인공 떡대미남수
수도 은근 자각없이 플러팅하는거같은데 아무리 의구심때문에 신체적 접촉을 한거긴 호감도 없이 글케 스킨쉽을
호연이 뺨친 얼굴 치명적이라 명치가 아프다
가슴이 시끄러워요
참아야 하는거 알지만 벽고 번만
마뽀 우짜노
아자씨 귀여워요
문성씨 코트 안에 입고 활동하시는 거예요
육체미 난리났다
침대 구깃한거보고 얼굴 붉히냐고 새파랄게 어린것한테 동했다고 말을 못해
줄 1, 열 1 1,560,294자
```



28899 씩땡
28900 씩덴
28901 씩달
28902 씩벨
28903 씩팔
28904 씩창
28905 씩8
28906 씩ㅂㅏ
28907 씩가랭넌
28908 씩가랭년
28909 씩가랭눔
28910 씩겅
28911 씩댕
28912 씩댕미
28913 씩댕
28914 씩바라
28915 씩바알
28916 씩박색희
28917 씩박색히
28918 씩박색히
28919 씩말년
28920 씩말름
28921 씩말뎡신
28922 씩방새
28923 씩방세
28924 씩뺑가리
28925 씩버럼
28926 씩벌년
28927 씩벌쉐미
28928 씩벌텡
28929 씩벨
28930 씩볼텡
28931 씩부할
28932 씩부랄
28933 씩부렐
28934 씩불알
28935 씩뷰렐
28936 씩뷰렐
28937 씩브렐
28938 씩블년
28939 씩블



Python의 timeit 으로 측정

출처 및 참고 자료 : 속도 분석 + 성능 비교

<https://iostream.tistory.com/144>

한국어 분석 도구들

. KoNLPy : 다양한 형태소 분석기를 제공하는 파이썬 패키지. Okt(Open Korea Text), Hannanum, Kkma, Komoran, Mecab 등 포함 됨.

. Okt : 트위터분석기라는 이름으로 진행되었던 프로젝트가 fork되어 진행되는 것으로 기본 베이스는 트위터 분석기

. Hannanum : KAIST에서 개발한 형태소 분석기

. Mecab : 일본어 형태소 분석기를 한국에 맞게 수정한 버전.

. Komoran : Java로 작성된 형태소 분석기,

. Soynlp : 비지도 학습을 기반으로 하는 한국어 전용 토큰화 도구. 데이터 내에서 자주 등장하는 단어를 기반으로 하는 토큰화를 수행.

. Kkma : 서울대학교에서 만든 분석기, 성능은 좋으나 속도면에서 단점이 있다.

. Khaiii : 카카오에서 개발한 형태소 분석기. 딥러닝 기반으로 정확도가 높음.

. KSS(Korean Sentence Splitter) : 한국어 문장 분리를 위한 도구. 문장의 경계를 잘 인식하여 텍스트를 문장 단위로 나누는데 유용.

. ETRI : 한국전자통신연구원에서 개발한 한국어 처리 API. 형태소 분석, 개체명 인식, 의존 구문분석 등 다양한 기능을 제공. 인터넷 연결이 필요.

. Pororo : 카카오브레인에서 만든 한글 자연어처리 라이브러리.

. KoBERT : 한국어 정보 처리를 위해 특별히 훈련된 BERT 모델. 토큰화 기능 외에도 한국어 텍스트의 이해와 생성 작업에 탁월한 성능을 보임.

. Kiwi : 다양한 자연어 처리 기능을 제공. 형태소 분석, 품사 태깅 등 지원.

. KoGPT-2 : 한국어 생성 모델, 자연어 이해 및 생성 작업에 사용됨. 대화 생성, 문장 완성 등의 작업에 적합.

리스트 대신
numpy 씬

```
import pandas as pd
import numpy as np
import matplotlib.font_manager as fm
import matplotlib.pyplot as plt
import re
from tqdm import tqdm

!apt -qq -y install fonts-nanum > /dev/null

# 데이터 불러오기
train_data = pd.read_csv('/content/drive/MyDrive/boom_final.csv')
print('데이터 개수:', len(train_data))

# 문자열이 아닌 데이터 제거
train_review = train_data['댓글'].dropna().map(str)

# 문자열이 아닌 데이터를 모두 제거한 후 numpy 배열로 변환합니다.
train_review = np.array([review for review in train_data['댓글'] if type(review) is str])

# 불용어 목록 파일을 불러옵니다.
with open('/content/drive/MyDrive/stopword_main.txt', 'r', encoding='utf-8') as f:
    stopwords = np.array([line.strip().replace('₩r', '') for line in f.readlines()])

# 불용어를 제거하고, 대소문자 및 특수문자 처리합니다. (리스트 컴프리헨션을 사용하여 처리)
filtered_review = []
for review in tqdm(train_review, desc="리뷰 처리 중"):
    review = review.lower()
    review = re.sub(r'[^가-힣₩s]', '', review)
    words = review.split()

    # numpy 배열을 사용하여 불용어를 필터링합니다. 이 경우, 리스트 컴프리헨션을 사용하는 것이 더 적합합니다.
    filtered_words = ' '.join([word for word in words if word not in stopwords])
    filtered_review.append(filtered_words)
```

```
# !pip install konlpy 인스톨하세요

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re
from tqdm import tqdm
from konlpy.tag import Okt

# 형태소 분석기 초기화
okt = Okt()

# 데이터 불러오기
data = pd.read_csv('/content/AllReviewCorpus_Moon.csv')
print('데이터 개수:', len(data))

# 문자열이 아닌 데이터 제거
data_1 = data['댓글'].dropna().map(str)

# 불용어 목록 파일을 불러옵니다.
with open('/content/stopword_ko_Main.txt', 'r', encoding='utf-8') as f:
    stopwords = np.array([line.strip().replace('₩r', '') for line in f.readlines()])

# 명사 추출 및 불용어 처리
filtered_review = []
for review in tqdm(data_1, desc="리뷰 처리 중"):
    review = re.sub(r'[^ㄱ-ㅎㅌ-ㅣ가-힣a-zA-Z0-9]+', '', review)
    nouns = okt.nouns(review) # 형태소 분석을 통해 명사만 추출

    # numpy 배열을 사용하여 불용어를 필터링합니다. 이 경우, 리스트 컴프리헨션을 사용하는 것이 더 적합합니다.
    filtered_nouns = ' '.join([noun for noun in nouns if noun not in stopwords])
    filtered_review.append(filtered_nouns)
```

```
!pip install konlpy
```

```
import pandas as pd
import numpy as np
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import re
from tqdm import tqdm
from konlpy.tag import Hannanum # 한나눔으로 변경
```

```
!apt -qq -y install fonts-nanum > /dev/null
```

```
# 형태소 분석기 초기화 (한나눔으로 변경)
hannanum = Hannanum()
```

```
# 데이터 불러오기
train_data = pd.read_csv('/content/코퍼스데이터.csv')
print('데이터 개수:', len(train_data))
```

```
# 문자열이 아닌 데이터 제거
train_review = train_data['댓글'].dropna().map(str)
```

```
# 불용어 목록 파일을 불러옵니다.
with open('/content/drive/MyDrive/stopword_main.txt', 'r', encoding='utf-8') as f:
    stopwords = np.array([line.strip().replace('₩r', '') for line in f.readlines()])
```

```
# 명사 추출 및 불용어 처리 (한나눔 사용)
filtered_review = []
for review in tqdm(train_review, desc="리뷰 처리 중"):
    review = review.lower()
    review = re.sub(r'[^ㄱ-ㅎ|ㅏ-ㅣ|가-힣|a-zA-Z0-9]+', '', review)
    nouns = hannanum.nouns(review) # 한나눔 형태소 분석을 통해 명사만 추출
```

```
# numpy 배열을 사용하여 불용어를 필터링합니다. 이 경우, 리스트 컴프리헨션을 사용하는 것이 더 적합합니다.
filtered_nouns = ''.join([noun for noun in nouns if noun not in stopwords])
filtered_review.append(filtered_nouns)
```

```
!pip install mecab-python #메캅 : 일본어를 형태소 분석기를 한국 버전으로 수정한 것
!pip install install curl git
!bash <(curl -s https://raw.githubusercontent.com/konlpy/konlpy/master/scripts/mecab.sh)
!pip install konlpy #코엔얼파이 : 한국어 자연어 처리 패키지
```

```
from konlpy.tag import Mecab
mecab = Mecab()
print(mecab.nouns('메캅이 잘 설치되었는지 확인하는 테스트 문장입니다.'))
```

```
import pandas as pd
import numpy as np
from wordcloud import WordCloud
import matplotlib.pyplot as plt
import re
from tqdm import tqdm
from konlpy.tag import Mecab
```

```
!apt -qq -y install fonts-nanum > /dev/null
```

```
# 형태소 분석기 초기화
mecab = Mecab()
```

```
# 데이터 불러오기
train_data = pd.read_csv('/content/코퍼스데이터.csv')
print('데이터 개수:', len(train_data))
```

```
# 문자열이 아닌 데이터 제거
train_review = train_data['댓글'].dropna().map(str)
```

```
# 불용어 목록 파일을 불러옵니다.
with open('/content/drive/MyDrive/stopword_main.txt', 'r', encoding='utf-8') as f:
    stopwords = np.array([line.strip().replace('\r', '') for line in f.readlines()])
```

```
# 명사 추출 및 불용어 처리
filtered_review = []
for review in tqdm(train_review, desc="리뷰 처리 중"):
    review = review.lower()
    review = re.sub(r'[\^~!@#$%^&*|+-|=|가-힣|a-zA-Z0-9|+]', '', review)
    nouns = mecab.nouns(review) # 형태소 분석을 통해 명사만 추출
```

```
# numpy 배열을 사용하여 불용어를 필터링합니다. 이 경우, 리스트 컴프리헨션을 사용하는 것이 더 적합합니다.
filtered_nouns = ' '.join([noun for noun in nouns if noun not in stopwords])
filtered_review.append(filtered_nouns)
```

```
!git clone https://github.com/kakao/khaiii.git
!pip install cmake
!mkdir build
!cd build && cmake /content/khaiii
!cd /content/khaiii/build/ && make all
!cd /content/khaiii/build/ && make resource
!cd /content/khaiii/build/ && make install
!cd /content/khaiii/build/ && make package_python
!pip install /content/khaiii/build/package_python
```

```
from khaiii import KhaiiiApi
import pandas as pd
import numpy as np
import re
from tqdm import tqdm

# 형태소 분석기 초기화
api = KhaiiiApi()

# 데이터 불러오기
train_data = pd.read_csv('/content/코퍼스데이터.csv')
print('데이터 개수:', len(train_data))

# 문자열이 아닌 데이터 제거
train_review = train_data['댓글'].dropna().map(str)

# 불용어 목록 파일을 불러옵니다.
with open('/content/drive/MyDrive/stopword_main.txt', 'r', encoding='utf-8') as f:
    stopwords = np.array([line.strip().replace('\r', '') for line in f.readlines()])

# 명사 추출 및 불용어 처리
filtered_review = []
for review in tqdm(train_review, desc="리뷰 처리 중"):
    review = review.lower()
    review = re.sub(r'^\n-~|~|~|가-형[a-zA-Z0-9]+', '', review)
    nouns = []
    for word in api.analyze(review):
        for morph in word.morphs:
            if morph.tag in ['NNG', 'NNP']: # 일반 명사와 고유 명사만 추출
                nouns.append(morph.lex)

    filtered_nouns = ' '.join([noun for noun in nouns if noun not in stopwords])
    filtered_review.append(filtered_nouns)
```

```
!pip install pororo

from pororo import Pororo
import pandas as pd
import numpy as np
import re
from tqdm import tqdm

# 형태소 분석기 초기화
nlp = Pororo(task="pos", lang="ko")

# 데이터 불러오기
train_data = pd.read_csv('/content/코퍼스데이터.csv')
print('데이터 개수:', len(train_data))

# 문자열이 아닌 데이터 제거
train_review = train_data['댓글'].dropna().map(str)

# 불용어 목록 파일을 불러옵니다.
with open('/content/drive/MyDrive/stopword_main.txt', 'r', encoding='utf-8') as f:
    stopwords = np.array([line.strip().replace('#r', '') for line in f.readlines()])

# 명사 추출 및 불용어 처리
filtered_review = []
for review in tqdm(train_review, desc="리뷰 처리 중"):
    review = review.lower()
    review = re.sub(r'[^ㄱ-ㅎ|ㅏ-ㅣ|가-힣|a-zA-Z0-9]+', '', review)
    pos_result = nlp(review)
    nouns = [word[0] for word in pos_result if word[1] in ['NNG', 'NNP']] # 일반 명사와 고유 명사만 추출

    filtered_nouns = ' '.join([noun for noun in nouns if noun not in stopwords])
    filtered_review.append(filtered_nouns)
```

```
# !apt -qq -y install fonts-nanum > /dev/null #한글 글꼴 설치
# !pip install wordcloud
```

```
import pandas as pd
import numpy as np
from wordcloud import WordCloud
import matplotlib.pyplot as plt
```

```
data = pd.read_csv('/content/filtered_review.csv') #워드클라우드 파일 불러오기
```

```
# 다시 numpy 배열로 변환합니다.
filtered_review = np.array(filtered_review)
```

```
# 워드 클라우드 생성 및 출력
```

```
fontpath = '/usr/share/fonts/truetype/nanum/NanumBarunGothic.ttf'
```

```
wordcloud = WordCloud(font_path=fontpath, background_color='white', width=800, height=800, stopwords=set(stopwords)).generate(' '.join(filtered_review))
plt.figure(figsize=(8, 8))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()
```

프로젝트 수행 결과

각 WordCloud 결과



불용어 처리



mecab + 불용어 처리



okt + 불용어 처리



hannanum + 불용어 처리

```
1 pip install pororo
```

Collecting pororo

Using cached pororo-0.4.2-py3-none-any.whl (256 kB)

INFO: pip is looking at multiple versions of pororo to determine which version is compatible with other requirements. This could take a while.

Using cached pororo-0.4.1-py3-none-any.whl (253 kB)

Using cached pororo-0.4.0-py3-none-any.whl (253 kB)

Using cached pororo-0.3.5-py3-none-any.whl (253 kB)

Using cached pororo-0.3.4-py3-none-any.whl (216 kB)

Using cached pororo-0.3.3-py3-none-any.whl (215 kB)

Using cached pororo-0.3.2-py3-none-any.whl (212 kB)

Using cached pororo-0.3.1-py3-none-any.whl (211 kB)

INFO: pip is looking at multiple versions of pororo to determine which version is compatible with other requirements. This could take a while.

ERROR: Cannot install pororo==0.3.1, pororo==0.3.2, pororo==0.3.3, pororo==0.3.4, pororo==0.3.5, pororo==0.4.0, pororo==0.4.1 and pororo==0.4.2 because these package versions have conflicting dependencies.

The conflict is caused by:

pororo 0.4.2 depends on torch==1.6.0

pororo 0.4.1 depends on torch==1.6.0

pororo 0.4.0 depends on torch==1.6.0

pororo 0.3.5 depends on torch==1.6.0

pororo 0.3.4 depends on torch==1.6.0

pororo 0.3.3 depends on torch==1.6.0

pororo 0.3.2 depends on torch==1.6.0

pororo 0.3.1 depends on torch==1.6.0

To fix this you could try to:

1. loosen the range of package versions you've specified

2. remove package versions to allow pip attempt to solve the dependency conflict

#Pip# ResolutionImpossible: for help visit <https://pip.pypa.io/en/latest/topics/dependency-resolution/#dealing-with-dependency-conflicts>

```
CMake Error at /root/.hunter/_Base/Download/Hunter/0.23.34/70287b1/Unpacked/cmake/modules/hunter_wiki.cmake:12 (message):
Call Stack (most recent call first):
  /root/.hunter/_Base/Download/Hunter/0.23.34/70287b1/Unpacked/cmake/modules/hunter_fatal_error.cmake:20 (hunter_wiki)
  /root/.hunter/_Base/Download/Hunter/0.23.34/70287b1/Unpacked/cmake/modules/hunter_download.cmake:613 (hunter_fatal_error)
  /root/.hunter/_Base/Download/Hunter/0.23.34/70287b1/Unpacked/cmake/projects/GTest/hunter.cmake:244 (hunter_download)
  /root/.hunter/_Base/Download/Hunter/0.23.34/70287b1/Unpacked/cmake/modules/hunter_add_package.cmake:62 (include)
  CMakeLists.txt:63 (hunter_add_package)
```

-- Configuring incomplete, errors occurred!

/bin/bash: line 1: cd: /content/khaiii/build/: No such file or directory

/bin/bash: line 1: cd: /content/khaiii/build/: No such file or directory

/bin/bash: line 1: cd: /content/khaiii/build/: No such file or directory

/bin/bash: line 1: cd: /content/khaiii/build/: No such file or directory

ERROR: Invalid requirement: '/content/khaiii/build/package_python'

Hint: It looks like a path. File '/content/khaiii/build/package_python' does not exist.

```
!pip install konlpy
```

```
from konlpy.tag import Okt
okt = Okt()
```

```
import pandas as pd
DATA_PATH = '/content/'
filtered_review = pd.read_csv(DATA_PATH+'filtered_review.csv')
```

```
import re
f = open('/content/stopword_ko_Main.txt')
stop_words = f.read().split()
def preprocessing_vec(review, okt): # 불러온 데이터셋은 이미 잘 전처리가 되어 있지만 마지막 더블체크 수행.
    # 1. 정규화 : 한글 및 공백을 제외한 문자 모두 제거.
    review_text = re.sub("[^가-힣\s0-9]", "", review)

    # 2. 형태소 : okt 형태소 토큰화 + 품사 태깅
    word_review = okt.pos(review_text, stem=True)

    # 노이즈 & 불용어 제거
    word_review = [(token, pos) for token, pos in word_review if not token in stop_words and len(token) > 1]

    # 명사 추출
    word_review = [token for token, pos in word_review if pos in ['Noun']]

    return word_review
```

```
from tqdm import tqdm
clean_review = []
for review in filtered_review['filtered_review']:

    if type(review) == str:
        clean_review.append(preprocessing_vec(review,okt))
    else:
        clean_review.append([])
```

```
clean_review = sum(clean_review, [])
```

```
clean_review = pd.DataFrame(clean_review)
```

```
clean_review[0]
```

```
clean_review.to_csv("clean_review.csv", index = False)
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.datasets import fetch_20newsgroups
```

```
vectorizer = TfidfVectorizer(max_features= 1000, # 상위 1,000개의 단어를 보존
                              max_df = 0.5, smooth_idf=True)
```

```
X = vectorizer.fit_transform(clean_review[0])
```

```
# TF-IDF 행렬의 크기 확인
print('TF-IDF 행렬의 크기 :',X.shape)
```

```
from sklearn.decomposition import LatentDirichletAllocation
# 모델 객체 생성
lda_model = LatentDirichletAllocation(n_components=20, random_state=777)
```

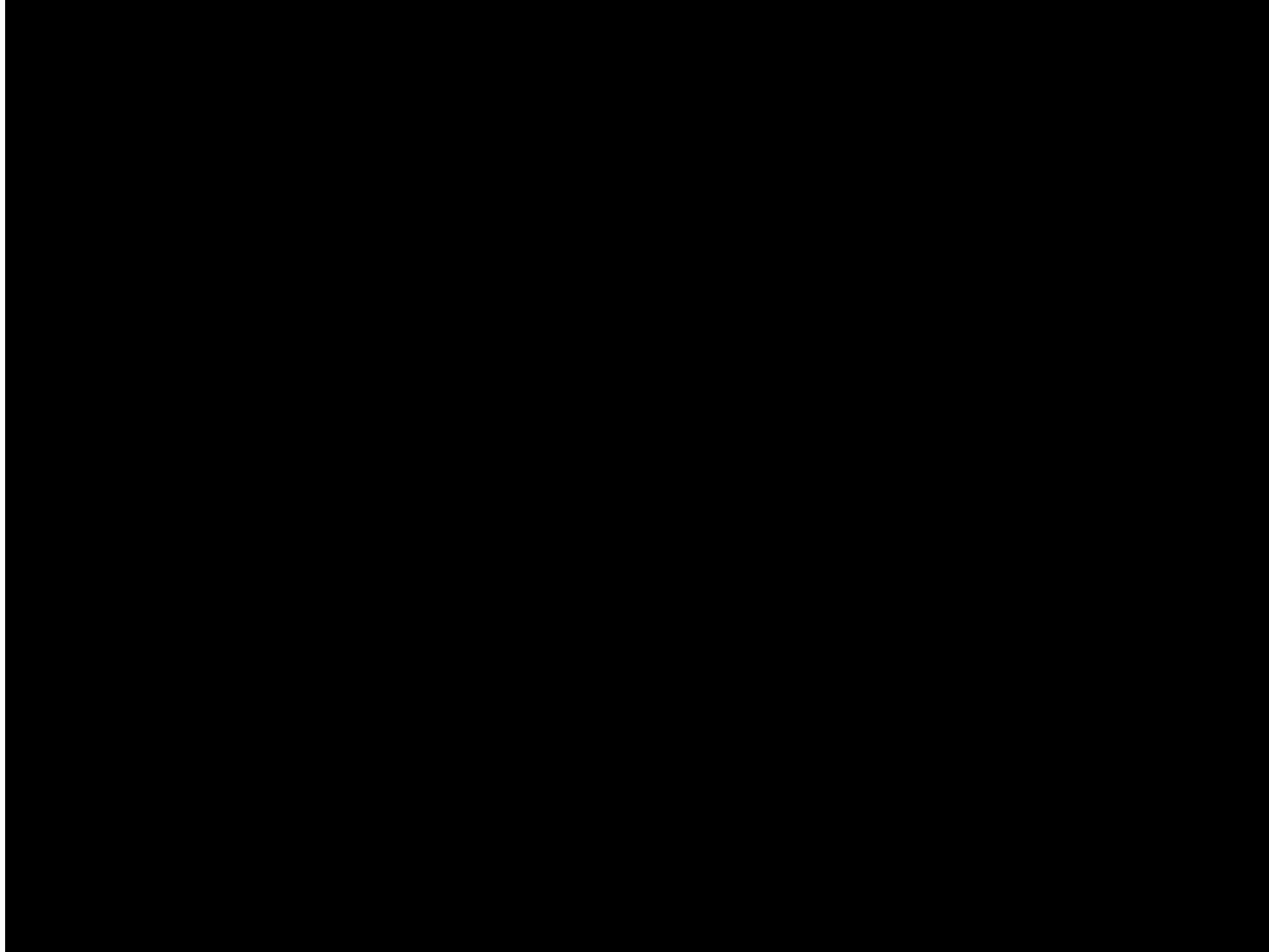
```
# fit transform
lda_model.fit(X)
```

```
# 단어 집합. 1,000개의 단어가 저장됨.
terms = vectorizer.get_feature_names_out()
```

```
def get_topics(components, feature_names, n=5):
    for idx, topic in enumerate(components):
        print("Topic %d:" % (idx+1), [(feature_names[i], topic[i].round(2)) for i in topic.argsort()[::-n - 1:-1]])
```

```
get_topics(lda_model.components_, terms)
```

```
# 시각화
# LDavis 는 토픽 모델링에 자주 이용되는 Latent Dirichlet Allocation (LDA) 모델의 학습 결과를 시각적으로 표현하는 라이브러리입니다
!pip install -U pyLDavis
```



Translator ()

```
!pip install googletrans==4.0.0-rc.1

from googletrans import Translator
import pandas as pd
import os
from tqdm import tqdm

'''
AllReviewCorpus_Moon
해당 파일은 깃허브 파일 기준 csv 파일입니다.
'''

local=os.getcwd()

ppd=pd.read_csv(local+'//AllReviewCorpus_Moon.csv')
tran = Translator()
#translator.translate('안녕하세요.')
# <Translated src=ko dest=en text=Good evening. pronunciation=Good evening.>

def Tran_KRtoEN(txt,c,y):
    '''
        txt : pandas.core.series.Series 컬럼 인수값
        y   : 원하는 컬럼 갯수

        반환값 : list형식
    '''
    krenStr=[]
    for i in tqdm(range(c,y)):
        try:
            kkr=tran.translate(txt[i], dest='en')
            #print(kkr.text)
            krenStr.append(kkr.text)
```

프로젝트 수행 결과

한국어 코퍼스 영문 전환 결과

	A	B	C	D	E	F		A	B	C	D	E	F	G	H	I												
1	아니.... 상상도 못한 방법으로 회피하셔가지고 저도 놀랐어요...						1	No	I	was	surprised	to	avoid	it	in	an	unimaginable	way										
2	아무리 생각해도 어릴적에 너무나도 매력적인 남성과의 접촉이						2	No	matter	how	much	you	think	about	it	it	may	be	that	your	contact	with	a	man	who	is	so	at
3	하...계란야무지게엮은미니토스트아이팟케이스쓰는공어때,						3	Ha		when	I	write	a	mini	toast	iPod	case	with	eggs									
4	그럼 아저씨가 평생 책임져요						4	Then	the	uncle	is	responsible	for	the	lifetime													
5	정말 죄송한데 딱 한번만 실수로 벽에 끼이셨으면..!						5	Im	really	sorry	but	I	just	want	to	be	on	the	wall	by	mistake							
6	뽕갈하고 이마깨지기ㅋㅋㅋㅋ큐ㅠㅠ						6	Spring	and	breaching	the	forehead																
7	문성이 가슴에 있는 햄버거 먹으면 되겠네						7	You	can	eat	hamburgers	in	the	chest														
8	작가님 전 좋은 독자가 될게요						8	I	will	be	a	good	reader															
9	와 과거의 옆집 아저씨가 지금은 자신을 선생님이라고 부르고						9	And	the	uncle	next	to	the	past	is	crazy	that	he	is	calling	himself	a	teacher					
10	아기공 동정따먹고 감옥가도 할말없다는 연상수 ㅋㅋㅋㅋㅋㅋ						10	Baby	hoon	sympathy	and	prison	is	also	a	rating												
11	가슴이 시끄럽네요						11	My	heart	is	noisy																	
12	너는 무슨 같이 싸우겠다는 말을 뽀스만 입고 하니						12	Are	you	wearing	only	pants	saying	that	you	want	to	fight	together									
13	진심 미칠거같아ㅠㅠㅠㅠㅠㅠ 총 안맞을 수 있는데 공 차에 안						13	I	think	Im	going	crazy	You	may	not	be	able	to	hit	the	gun	but	its	not	right	in	the	ball
14	수 허벅지랑 공 허리가 비슷하다? 진짜 눈물이 입가에서 흐른다						14	Is	it	similar	to	the	thighs	and	the	ball	backReal	tears	flow	from	the	mouth						
15	내 거대 날다람쥐야....						15	My	huge	flying	squirrel																	
16	저런 엄한 몸매로 밥을 해주다니..결혼이죠 이건..						16	Youre	cooking	with	such	a	strict	body	Its	married	this	is										
17	술 약한 아기공 집에 데려다 줘 떨어지는 라면 봉지 잡아 줘 근						17	Take	it	to	the	weak	babys	house														
18	저렇게. 멀끔하고.가슴을 내미는데.고.백을.어떻게.거 절합니까..						18	Like	thatIts	farl	put	out	my	chestHow	do	you	pick	up	the	backFoxy	is	ordered						
19	와시 허리들어간거봐 옷 너무 바람직하다						19	Washis	waist	is	so	desirable																
20	아니 미친 문성씨 시계 찬거 왜 이렇게 꼴림? 진짜 싹. 댄디. 싹						20	No	why	are	you	so	crazyReal	bitterDandySweet	rice	cakelts	crazy	because	it	seems	to	give	a					
21	잠깐 좀 먼곳에 가있었다는 게 과거였던걸까요?미인공 떡대미						21	Was	it	the	past	that	there	was	a	little	longerI	like	beauty	rice	cake							
22	수도 은근 자각없이 플러팅하는거같은데 ㅋㅋㅋㅋㅋ 아무리 의						22	I	think	Im	fluttering	without	a	capital	but	I	have	physically	contacted	because	of	doubts	but					

프로젝트 수행 결과

영문_1만개 워드클라우드



작성 코드

```
local=os.getcwd()

example=pd.read_csv(local+'//csv//bomtoon_final.csv')
colname=list(example.keys())

# 추가할 영문 불용어 정의
#additional_stop_words = ['ha', 'im', 'tt', 'going', 'oh', 'wow', 'know', 'ah', 'dont', 'think', 'come', 'want', 'like', 'g

# scikit-learn에서 제공하는 영어 불용어에 사용자 정의 불용어를 합침
#stop_words = list(text.ENGLISH_STOP_WORDS.union(additional_stop_words))

example['0'].isna().sum() # 전체 컬럼에 NaN 확인

cv = CountVectorizer(max_features=1000, stop_words = stop_words ,ngram_range=(1,2)) # 문장별로 단어 행렬 변환을
tr_comment = cv.fit_transform(example[colname[1]].values.astype('U')) # 학습을 위해 fit_transform

words = cv.get_feature_names_out() # 결측치 무결성

doc = tr_comment[0].toarray()
```

추가된 불용어 미처리 및 테스트데이터 확인

프로젝트 수행 결과

영문_ALL댓글 워드클라우드

작성 코드



추가된 불용어 처리 및 전체 댓글

```
# 추가할 영문 불용어 정의
additional_stop_words = ['ha', 'im', 'tt', 'going', 'oh', 'wow', 'know', 'ah', 'dont', 'think', 'come', 'want', 'like', 'good']

# scikit-learn에서 제공하는 영어 불용어에 사용자 정의 불용어를 합침
stop_words = list(text.ENGLISH_STOP_WORDS.union(additional_stop_words))

example['col1'].sum() # 전체 컬럼의 NaN 확인

cv = CountVectorizer(max_features=1000, stop_words=stop_words, ngram_range=(1,2)) # 문장별로 단어 행렬 변환을 위해
tr_comment = cv.fit_transform(example[colname[1]].values.astype('U')) # 학습을 위해 fit transform 사용
```

TF-IDF 행렬의 크기 : (72635, 1000)

```
vectorizer = TfidfVectorizer(stop_words=stop_words, max_features=
1000,
# 상위 1,000개의 단어를 보존
max_df = 0.5, smooth_idf=True)
```

```
X = vectorizer.fit_transform(getRead_word['0'].values.astype('U'))
# astype()을 사용한 데이터형 dtype 변환 (캐스트)
```

```
# TF-IDF 행렬의 크기 확인
print('TF-IDF 행렬의 크기 :',X.shape)
```

```
·
·
·
```

```
pyLDAvis.enable_notebook()
vis = pyLDAvis.lda_model.prepare(
    lda_model = lda_model,
    dtm = X,
    vectorizer = vectorizer,
    mds='tsne')
pyLDAvis.display(vis)
```



Conclusion Insight



페이백(PAYBACK)

samk&푸조킹

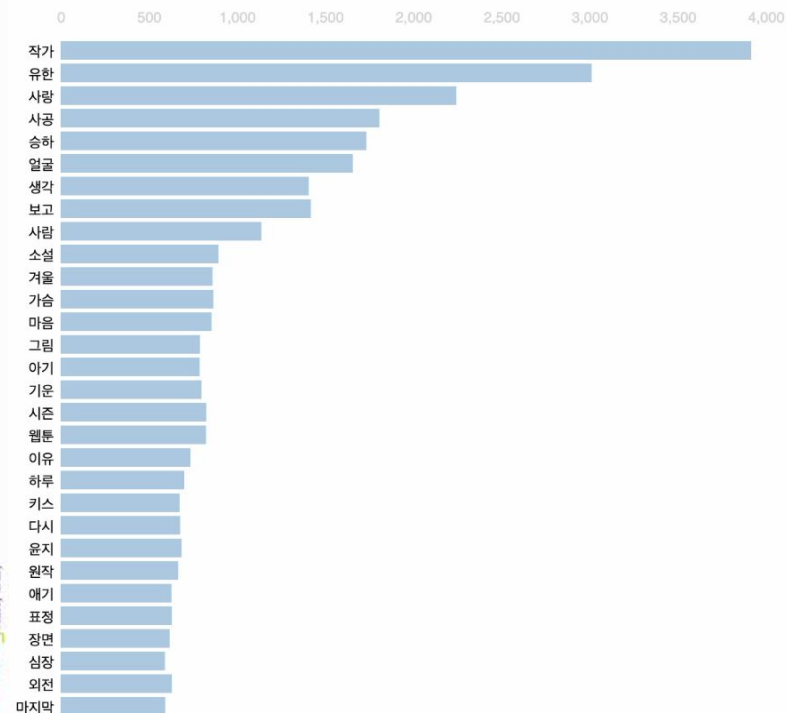
작품설명

남들은 수능을 준비하는 나이에 대부분 체에 들어가 돈을 받아내려 **협박과 행패**를 일삼으며 만나니 같은 삶을 살던 **유한**. 그러던 어느날, 그는 자신에게 앙심을 품은 이의 손에 **가죽을 잃게** 된다. 본인으로부터 비롯된 가족의 죽음에 죄책감을 느낀 유한은 그날 이후, 속죄하고자 하는 마음으로 고된 일만을 씬없이 하며 살아간다. 그러다 우연히, 그는 옛 동거인 송명신이 동생의 죽음에 일조했다는 사실을 알게 되고, **그에게 복수**를 결심한다.

그리고 그런 유한은 '도움을 주겠다'는 **남자와 마주하는데...**



Top-30 Most Salient Terms¹



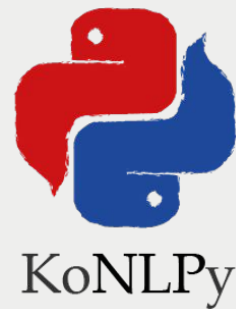
Overall term frequency
Estimated term frequency within the selected topic

1. saliency(term w) = frequency(w) * [sum_t p(t | w) * log(p(t | w)/p(t))]] for topics t; see Chuang et. al (2012)
2. relevance(term w | topic t) = λ * p(w | t) + (1 - λ) * p(w | t)/p(w); see Sievert & Shirley (2014)

What we used



PYTHON / Google Colab



KoNLPy



GitHub

LDA



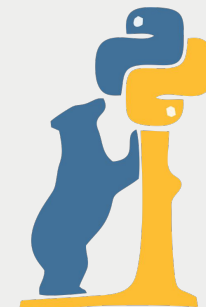
LDA / word cloud



NumPy

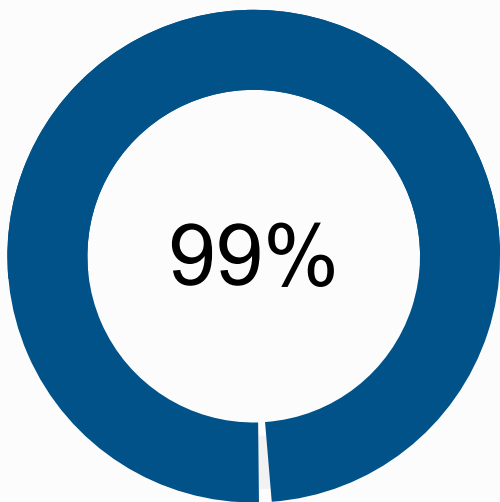
NumPy

Pandas

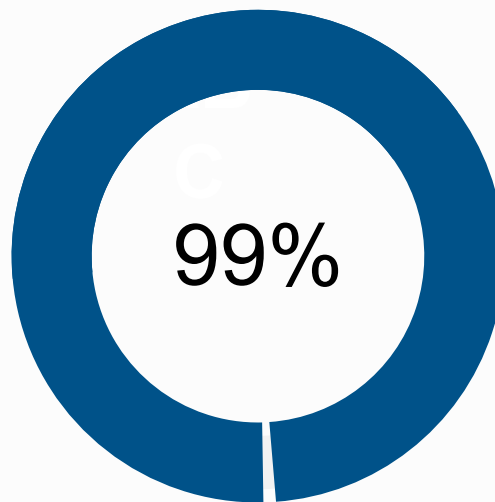


Pandas

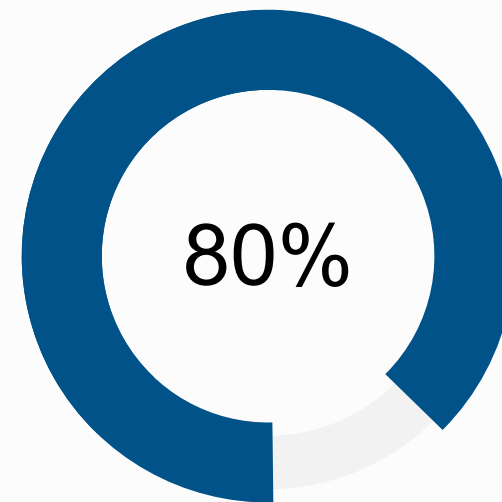
자체 평가 의견



기획 부합도



실무 활용도



달성도

- ◆ 데이터 크롤링 OK
- ◆ 텍스트 전처리 OK
- ◆ 워드 클라우드 OK
- ◆ LDA OK
- ◆ 페어플롯 Not Yet
- ◆ 감성분석 Not Yet

자체 평가

김예선 : 팀원분들과 협업해서 결과를 도출하는 경험이 큰 성장이 되었습니다. **LDA** 등 많이 배울 수 있어서 좋았습니다.

문영식 : 전반적인 진행과 협업에 있어서 만족도가 크지만 결과물은 아쉬움이 있다, 더 많은 기술을 실험 분석 이해 적용하고 싶었다.

조인준 : 각각의 개별 아이디어를 하나로 융합하여 수업 시간 배운 내용을 접목하는 것이 원활히 진행되어 뜻깊은 여정이었다.

최희범 : 서로 익숙하지 않은 분야에 대해 이해해가며, 지속적인 커뮤니케이션을 통해 프로젝트를 완성한 점에서 좋은 경험이었음.

감사합니다

Q & A